

## Is it still needed today?

All right, enough of 1980's. The important question is, with the advent of languages such as Java, is C++ still needed? The fact that C++ is still very popular should answer that question. Why? Read on we'll cover that later in this chapter.

## The killer feature (RAII)

C++ is the only widely used language in the industry which supports the feature of RAII. For those, who aren't familiar with what RAII is, it basically stands for Scope Bound Resource Management. The control C++ provides over the lifetime of the objects it generates is something that garbage collection or any other management method lacks. In fact, in other languages, a common problem encountered is that objects exist beyond their lifetime. In C++ the objects can be acquired as well as be released in a deterministic manner.

## What about performance?

C++ performs better than all recently made languages such as Python or Java, which despite having large libraries are no match for C++. The reasons for this unbeatable performance are brevity, modular programming and, of course, closeness to the hardware.

## Brevity? Modular programming?

**Brevity:** Brevity refers to compactness of code. In other words, how short is the code, measured as LOC (Lines of Code). The more lines of code the more a project costs and performance takes a hit as well. This is why many companies such as Adobe, Amazon, Apple, Facebook, Quora etc., all have some parts of their basic codes written in C++.

**Modular Programming:** One of the most important aspects of Software Engineering is the concept of Modularity, which basically means the ability to build different, independent modules and later link them together in a program during compilation or run time. C++ not only promotes modularity within itself, but also with other languages, such as Assembler or C.

## Syntax:

### Hello world

- `#include<iostream>`
- `Using namespace std;`